

Log

ar064 · 11 July 2026 · Draft

The program's notebook — the *why* and *what-happened* that the run-stamps do not capture. Append-only, timestamped, newest first. Relaxed and chronological by design; still a published page, so run numbers cite the record, never hand-typed. The goal and the queue live in the plan the operating policy in the night-shift.

Digest

Latest, for the 30-second morning read. Updated 2026-07-11 23:35 BST.

Goal exceeded: the forward-pass state clamp is the best stable recipe. exp064 tested the lead the queue pointed to — and it wins. Reading the forward pass showed the divergence is the exp-Euler $v_\infty = (...)/g_{\text{tot}}$ blowing up when signed weights drive $g_{\text{tot}} = g_L + g_e + g_i \leq 0$ (why exp060 NaN'd at epoch 2 with tiny weights: it is the *sign*, not the magnitude). A clamp that floors conductances at 0 each timestep (physical) keeps $g_{\text{tot}} \geq g_L > 0$. Result: the free net goes from **13/30 NaN** → **0/30**, gradient $3 \cdot 10^4 \rightarrow \approx 190$, at **56.9%** — matching the free baseline (56.8%) and beating Dale's law (53.2%). It bounds the **state**, not the weights (W_{ee} unchanged), so it keeps the signed net's full accuracy where Dale's law paid for stability. **The free net never needed constraining, only its state bounding.** Implemented in `tools/snn/models.py` (`--state-clamp`, off by default).

The stability picture, ranked: state clamp (56.9%, stable) > Dale's law (53.2%, stable) $\gg$ free / Δt / weight decay (all diverge). Of the three soft knobs the free net can vary without a model change — Δt , Dale's law, weight decay — only Dale's law stabilises.

- **Δt (exp061, killed):** finer Δt made the gradient explosion *worse* (longer BPTT unroll), NaN persists at 0.25. Not stiffness.
- **Dale's law (exp062, supported):** the free net's 13/30 NaN epochs → **0/30** under the non-negativity constraint; gradient $3 \cdot 10^4 \rightarrow \approx 190$, W_{ee} 9.2 → 5.4. Costs accuracy (56.8% → 53.2%). **The stable recipe.**
- **Weight decay (exp063, killed):** NaN persists at every λ up to 1e-1 (15/13/12/11 of 30), W_{ee} unbounded (9 throughout). Regularises — accuracy rises to 63.4% — but does not stabilise.

Where the program goes next. Stabilising the *free* net (to keep its higher accuracy) is not reachable by any soft knob, so the reserved **forward-pass state clamp** — a shared-model change — is the live lead. exp063's accuracy headroom (63.4% at strong decay, above every other recipe) suggests a clamped-at-strong-decay signed net could top the program. **That is a model change and a human call — parked for the gate.**

Infra (resolved). SSH-blocked sandbox → rsync-over-SSH collector fails; fixed by reading the RunPod volume over its **S3 HTTPS API** (`collect_via_s3`).

Sessions

2026-07-11 23:35 BST — exp064: the state clamp wins (goal exceeded)

Implemented the reserved model change and ran it — it is the best recipe.

- **Forward-pass state clamp (`tools/snn/models.py`, `--state-clamp`).** Reading the COBA exp-Euler step showed the divergence mechanism: v_∞ divides by $g_{\text{tot}} = g_L + g_e + g_i$, and signed weights let a conductance go negative, so g_{tot} crosses zero and $v_\infty \rightarrow \text{NaN}$. The clamp

floors conductances at 0 (caps magnitude at 100 μ S) each timestep, keeping $g_{\text{tot}} \geq g_L > 0$. Off by default, so every prior run is unchanged.

- **exp064 — free vs clamp vs clamp+decay (done, stabilises).** Result: free 13/30 NaN $\rightarrow$ free+clamp **0/30**, gradient $3 \cdot 10^4 \rightarrow \approx 190$, at **56.9%** (vs free 56.8%). Both clamped cells stable; clamp beats Dale’s law (53.2%) with no accuracy cost. W_{ee} is unchanged by the clamp (9.1) — it bounds the state, not the weight, which is why weight decay could not fix it and the clamp can. Mechanism confirmed.
- **Operational note.** The 4090 pool in EU-RO-1 stalled two clamp pods in startup (no `config.json` after 40–80 min, no output); the same cell ran fine on a 5090. Killed the stalled pods (nothing leaked) and re-fired on 5090. A 4090 startup-stall flag for the next shift.

2026-07-11 20:45 BST — exp063: weight decay regularises, does not stabilise

Closed out the no-code-change stability queue.

- **exp063 — weight-decay sweep on the free net (done, kill).** Swept $\lambda \in \{0, 1e-3, 1e-2, 1e-1\}$ on *-no-dales-law* at $\Delta t = 1.0$. Result: NaN persists at every strength (15/13/12/11 of 30, never zero) and W_{ee} stays pinned near 9 regardless of λ — decay does not bound the runaway weight. Kill fires. But accuracy climbs monotonically with λ (56.3% $\rightarrow$ 63.4%, the free net’s best), so decay is a strong regulariser, not a stabiliser.
- **Program state.** The three soft knobs are exhausted; only Dale’s law stabilises. To stabilise the free net *and* keep its accuracy, the plan’s reserved forward-pass state clamp (a `tools/snn/models.py` change) is the next experiment — deliberately left for the human gate, since it is a model change, not a knob.

2026-07-11 20:35 BST — exp062: Dale’s law is the stabiliser (supported)

Ran the second stability probe; the program’s goal is met.

- **exp062 — Dale’s law free vs constrained (done, supported).** Two cells, identical but for the constraint, at $\Delta t = 1.0$ where the free net diverges. Result: the free net NaNs 13/30 epochs (peak gradient $\approx 3 \cdot 10^4$, W_{ee} 9.2); the Dale’s-law net trains **NaN-free** (0/30, gradient ≈ 190 , W_{ee} 5.4) at 53.2% vs the free net’s 56.8%. The non-negativity projection bounds the $E \rightarrow I \rightarrow E$ loop gain below the runaway threshold, so the divergence never starts. First stable recipe achieved.
- **Direction.** The remaining queue changes meaning. exp063 (weight decay) and the state clamp are no longer needed *for stability* — Dale’s law provides it — but they are the way to ask whether the free signed net’s extra accuracy can be kept without its divergence. That is the more interesting question now.

2026-07-11 20:10 BST — exp061 killed the Δt hypothesis; RunPod collect fixed

Ran the first stability experiment and solved a compute blocker on the way.

- **exp061 — Δt sweep (done, kill).** Swept $\Delta t \in \{1.0, 0.5, 0.25\}$ ms on the free signed-recurrent net, single seed, full exp060 scale, on RunPod. The result refutes the plan’s hypothesis: NaN is reproduced at the coarse Δt (so the mechanism is real) but **not removed** by finer Δt — it persists at $\Delta t = 0.25$, and the peak pre-clip gradient norm explodes monotonically worse ($3 \cdot 10^4 \rightarrow 5 \cdot 10^{12} \rightarrow 4 \cdot 10^{17}$) because quartering Δt quadruples the

BPTT unroll (1000 $\rightarrow$ 4000 steps). Kill criterion fires; the divergence is a gradient explosion over the recurrent unroll, not exp-Euler stiffness. Δt leaves the recipe.

- **Instrumentation.** Added two per-epoch metrics to the trainer the sweep needed but that were previously invisible: `grad_norm_max` (peak pre-clip global norm; only the mean was recorded) and `nan_forward_batches` (NaN forward passes were silently skipped). Additive only.
- **Compute blocker, fixed.** The RunPod fan-out’s collector rsyncs off the shared volume over SSH; the cloud sandbox blocks outbound :22, so pods trained fine but results were stranded on the volume. Routed around it: a RunPod network volume is S3-compatible over HTTPS, so `collect_via_s3` reads the trained cells straight off the volume with the S3 API. The earlier “failed” plumbing pods turned out to have trained correctly all along — only collection was broken.

2026-07-11 17:36 BST — program opened

Stood the program up end to end in one sitting.

- **Built the substrate.** SHD training now works through the CLI: an event-based loader (official train/test split, lazy binning to the model’s dt), all four recurrent blocks made trainable (the `W_ii` CLI path was a latent dead flag, now fixed), and an AdamW weight-decay knob. The data-look entry confirmed the dataset is clean and class-separable.
- **Ran the first training smoke** (exp060, Rung A: free signed-recurrent ceiling, dt 1.0, 1000-sample subset). It trains — loss 3.18 $\rightarrow$ 0.56, best test accuracy 61% against a 5% floor — so the pipeline is sound. But the free dynamics are unstable: scattered epochs return NaN train/test loss, pre-clip gradient norms spike to millions, and the E $\rightarrow$ E recurrent weight grows without bound.
- **Chased the instability and was wrong once.** First guess was unbounded weight growth; added weight decay. It tamed the gradient explosion (7.5M $\rightarrow$ 900) and lifted final accuracy (50% $\rightarrow$ 58%), but did *not* bound `W_ee` at 1e-3 and made the NaN *more* frequent — and a NaN showed up at epoch 2 with tiny weights. So the root cause is not weight size; it is an intermittent forward-pass divergence of the signed conductance dynamics, most likely exp-Euler stiffness at coarse dt. Parked for exp061.
- **Reframed the program.** What began as debugging is the actual goal: a stable training recipe, with each ingredient attributed. Queued exp061 (dt sweep), exp062 (Dale’s law as the implicit stabiliser), exp063 (weight-decay sweep), with a forward-state clamp held in reserve.